

CG2271 Real Time Operating Systems
2024/25 Semester 1
Final Assessment
SOLUTIONS

1. (8 MARKS) Process Scheduling

In this question we look at rate-monotonic scheduling (RMS) versus fixed-priority scheduling where the priorities are assigned independently of the process period.

We have three periodic processes in a fixed priority scheme with a pre-emptive scheduler, with the following characteristics. CPU Time is the time that the process runs each time it is started up (excluding interrupts and pre-emptions by higher priority processes).

A process must complete its run before the end of its period. Otherwise it is considered to have missed its deadline.

Process	CPU Time	Period	Priority
P1	100 us	500 us	Medium
P2	50 us	200 us	Low
P3	75 us	300 us	High

- a. (3 marks) What is the worst-case response time (WCRT) of the 3 processes given the priorities above? Express your answers in us ($1 \text{ us} = 10^{-6} \text{ s}$)

Priority: P3, P1, P2

$$\begin{aligned}R_{31} &= 75 \\R_{11} &= 100 + \left\lceil \frac{100}{300} \right\rceil \times 75 \\&= 175 \\R_{12} &= 100 + \left\lceil \frac{175}{300} \right\rceil \times 75 \\&= 175 \\R_{21} &= 50 + \left\lceil \frac{50}{300} \right\rceil \times 75 + \left\lceil \frac{50}{500} \right\rceil \times 100 \\&= 50 + 75 + 100 \\&= 225 \\R_{22} &= 50 + \left\lceil \frac{225}{300} \right\rceil \times 75 + \left\lceil \frac{225}{500} \right\rceil \times 100 \\&= 225\end{aligned}$$

P1 WCRT = 175

P2 WCRT = 225

P3 WCRT = 75

- b. (3 marks) If the three processes are now scheduled using RMS, what is the WCRT of the three processes now? Again express your answers in us.

Under RMS, priority sequence is P2 P3, P1

$$\begin{aligned}R_{21} &= 50 \\R_{31} &= 75 + \left\lceil \frac{75}{200} \right\rceil \times 50 \\&= 125 \\R_{32} &= 75 + \left\lceil \frac{125}{200} \right\rceil \times 50 \\&= 125 \\P_{11} &= 100 + \left\lceil \frac{100}{200} \right\rceil \times 50 + \left\lceil \frac{100}{300} \right\rceil \times 75 \\&= 225 \\P_{12} &= 100 + \left\lceil \frac{225}{200} \right\rceil \times 50 + \left\lceil \frac{225}{300} \right\rceil \times 75 \\&= 100 + 100 + 75 \\&= 275 \\P_{13} &= 100 + \left\lceil \frac{275}{200} \right\rceil \times 50 + \left\lceil \frac{275}{300} \right\rceil \times 75 \\&= 275\end{aligned}$$

WCRT P1 = 275

WCRT P2 = 50

WCRT P3 = 125

- c. (2 marks) Choose ALL of the statements that are TRUE about RMS vs. fixed-priority scheduling in a pre-emptive operating system (Note: The term “schedulable” means that all processes will meet their deadlines).

[] Fixed-priority scheduling guarantees optimal assignment of priorities to processes.

[X] If a set of processes is not schedulable under RMS, there are no assignment of priorities in fixed-priority scheduling that can make the processes schedulable.

[] Any set of processes with a CPU utilization $U < 1$ is schedulable either under fixed-priority or RMS scheduling.

[X] Any set of processes that is schedulable given some priority assignment in a fixed-priority scheduler is guaranteed to be schedulable under RMS.

2. (10 MARKS) Semaphores

We are given the following semaphore operations:

Operation	Description
s = Semaphore(n);	Create a semaphore with initial value n.
s.take();	Decrement semaphore s if it is larger than 0, block otherwise.
s.give();	If any process is being blocked on s, causes one of the processes to be unblocked. Otherwise it increases the value of the semaphore.

In our system we have a sensor that periodically triggers interrupt INT1, and INT1's ISR will read the sensor and store it into a global variable d before triggering a process to put the reading in the global variable into a 6-element queue called Q. The queue is read by a second process.

We are using semaphores to coordinate between the ISR and processes, and to properly implement enqueue (enq) and dequeue (deq) functions.

Complete the program by choosing the appropriate statements for each of the blanks shown.

Options:

```
s1 = Semaphore(1);  
s2 = Semaphore(0);  
s3 = Semaphore(6);  
s1.take();  
s1.give();  
s2.take();  
s2.give();  
s3.take();  
s3.give();
```

```
#define QLEN      6
```

```
s1 = Semaphore(1);  
s2 = Semaphore(0);  
s3 = Semaphore(6);  
s4 = Semaphore(0);
```

```
volatile char d;
```

```
INT1_IRQHandler() {
```

```

        d = dev->data;
        s2.give();
    }

    char Q[QLEN];
    int head=0, tail=0;

    void enq(char data) {
        s3.take();
        s1.take();
        Q[head] = data;
        head = (head + 1) % QLEN;
        s1.give();
        s4.give();
    }

    char deq() {
        s4.take();
        s1.take();
        char t = Q[tail];
        tail = (tail + 1) % QLEN;
        s1.give();
        s3.give();
    }

    void process1() {
        // Process code
        while(1) {
            s2.take();
            enq(d);
        }
    }

    void process2() {
        while(1) {
            char d = deq();
            // Code to process d not shown.
        }
    }

```

3. (10 MARKS) Process Management

We will now explore the states of two processes with the following execution profiles, on a single core CPU with a single IO. The OS has a fixed priority pre-emptive scheduler, and P1 has the highest priority.

Process	Profile
---------	---------

P1	C2-IO1-C3-IO2-C1
P2	C4-IO2-C2

Here Cn means that the process uses the CPU for n time units, and IOm means that the process uses the IO for m time units. If a process wishes to use the IO and it is currently used by another process, it has to wait for the IO to become available and uses it as soon as possible.

Both processes are ready to run at time unit and terminate after the actions shown above. If it is a process's turn to run at time unit n, and it is ready, the process goes into a running state immediately.

Thus at time 0, the higher priority process will immediately be in a running state.

Complete the following table for the process states of P1 and P2 based on the information above:

Time	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
P1	Rn	Rn	Bl	Rn	Rn	Rn	Bl	Bl	Rn	Ex					
P2	Rd	Rd	Rn	Rd	Rd	Rd	Rn	Rn	Rd	Rn	Bl	Bl	Rn	Rn	Ex

4. (4 MARKS) Race Conditions

We are given the following two processes that access a shared variable x, whose initial value is 2:

Process P1

$x = x + 2;$

$x = x * 3;$

Process P2

$x = x - 1;$

How many UNIQUE values of x are there after both processes execute to completion?

Load and store instructions:

L11
S11
L12
S12

L21
S21

Possible interleavings:

$x=2$

$x = x + 2$

$x = * 3$

$x = x - 1$

L21 $x = 2$	L21 $x = 2$	L21 $x = 2$	L21 $x = 2$	L21 $x = 2$
S21 $x = 1$	L11 $x = 2$	L11 $x = 2$	L11 $x = 2$	L11 $x = 2$
L11 $x = 1$	S21 $x = 1$	S11 $x = 4$	S11 $x = 4$	S11 $x = 4$
S11 $x = 3$	S11 $x = 4$	S21 $x = 1$	L12 $x = 4$	L12 $x = 4$
L12 $x = 3$	L12 $x = 4$	L12 $x = 1$	S21 $x = 1$	S12 $x = 12$
S12 $x = 9$	S12 $x = 12$	S12 $x = 3$	S12 $x = 12$	S21 $x = 1$

L11 $x = 2$	L11 $x = 2$	L11 $x = 2$	L11 $x = 2$
L21 $x = 2$	L21 $x = 2$	L21 $x = 2$	L21 $x = 2$
S21 $x = 1$	S11 $x = 4$	S11 $x = 4$	S11 $x = 4$
S11 $x = 4$	S21 $x = 1$	L12 $x = 4$	L12 $x = 4$
L12 $x = 4$	L12 $x = 1$	S21 $x = 1$	S12 $x = 12$
S12 $x = 12$	S12 $x = 3$	S12 $x = 12$	S21 $x = 1$

L11 $x = 2$	L11 $x = 2$	L11 $x = 2$
S11 $x = 4$	S11 $x = 4$	S11 $x = 4$
L21 $x = 4$	L21 $x = 4$	L21 $x = 4$
S21 $x = 3$	L12 $x = 4$	L12 $x = 4$
L12 $x = 3$	S21 $x = 3$	S12 $x = 12$
S12 $x = 9$	S12 $x = 12$	S21 $x = 3$

L11 $x = 2$	L11 $x = 2$
S11 $x = 4$	S11 $x = 4$
L12 $x = 4$	L12 $x = 4$
L21 $x = 4$	L21 $x = 4$
S21 $x = 3$	S12 $x = 12$
S12 $x = 12$	S21 $x = 3$

L11 $x = 2$
S11 $x = 4$
L12 $x = 4$
S12 $x = 12$
L21 $x = 12$
S21 $x = 11$

Unique values are 12, 3, 1, 9, 11
There are 5 unique values.

5. (18 MARKS) Virtual Memory

We have a system with a 32-bit virtual address space and a 24-bit physical address space. Virtual pages (here simply called “pages”) and physical pages (here called “frames”) are 32 KiB long (1 KiB = 2^{10} bytes) long.

- a. (2 marks) How many bits are there in the page number and frame number portions of the virtual and physical addresses?

of bits in offset = $\log_2 32 \text{ KiB} = 15$
of bits in page number = $32 - 15 = 17$
of bits in frame number = $24 - 15 = 9$

- b. (2 marks) Assuming that each page table entry was 4 bytes, how much memory would the page table occupy. State your answer in KiB (1 KiB = 2^{10} bytes)

There are 2^{17} pages, each of size 2^2 . Total size = $2^{19} = 512 \text{ KiB}$

We now examine the impact of virtual memory on the time it takes to access an array. Assuming the same configuration as before (32-bit virtual addresses, 24-bit physical addresses and page/frame size of 32 KiB), we consider the following bit of code:

```
// Assume that integers are 4 bytes long
#define SIZE      262144

int A[SIZE];
int i, x;

for(i=0; i<SIZE; i++)
    x = A[i];
```

Assume that it takes 50 ns (1 ns = 10^{-9} seconds) to read/write each integer. For simplicity we will ignore the time taken to read/execute instructions, focusing only on the time to access the array A.

- c. (3 marks) Assuming that there is **NO VIRTUAL MEMORY**, How much time would it take to execute the for loop reading every element of array A? You may assume that the time taken to read/write variables i and x is negligible, and should not factor them into your calculation. Express your answer in ms (1 ms = 10^{-3} seconds) to two decimal places (e.g. 3.14)

We would be performing 262144 loads = $262144 \times 50 = 13.11 \text{ ms}$

- d. (3 marks) Assuming that i, x and A are stored in the same region of virtual memory, how many pages would they occupy in total?

$$\begin{aligned}\text{Total bytes} &= 4 + 262144 \times 4 = 1048580 \text{ bytes} \\ &= \lceil \frac{1048580}{32768} \rceil = 33 \text{ pages}\end{aligned}$$

- e. (4 marks) Now assuming that there is virtual memory and that the page table is also stored in memory, how long would it take to execute the for loop above to fully read array A? Again, you may assume that the time taken to read/write variables i and x is negligible and should not factor them into your calculations. State your answer in ms ($1 \text{ ms} = 10^{-3}$ seconds) to 2 decimal places (e.g. 3.14).

$$\begin{aligned}\text{Every memory access now requires a page table read and a read from the array.} \\ \text{Total time} &= 262144 \times 2 \times 50 = 26.21 \text{ ms}\end{aligned}$$

Let us now consider what happens when we introduce a translation lookaside buffer. Suppose that the TLB takes 2 ns to read. We assume that it takes negligible time to update the TLB.

- f. (4 marks) How much time will it now take for the loop above to execute to fully read array A? As before assume that the time taken to read/write the variables i and x is negligible, and you should not factor them at all into your calculations. State your answer in ms ($1 \text{ ms} = 10^{-3}$ seconds) to 2 decimal places (e.g. 3.14)

The first access to each page results in a TLB miss, and hence will take $2 + 50 = 52$ ns. So 33 such accesses will have a TLB/page table access time of 52 ns. The remaining 262111 accesses will have a TLB access time of 2 ns. All actual reads from the array will take 50 ns.

$$\begin{aligned}\text{Total} &= 33 \times 52 + 262111 \times 2 + 262144 \times 50 \\ &= 1716 + 524222 + 13107200 \\ &= 13.63 \text{ ms}\end{aligned}$$